

CS6004 PA4: Beat the Interpreter

Kritin Gupta (23B0948)
Gautam Siddharth K. (23B0957)

April 19, 2026

1 Introduction

We have implemented monomorphization and method inlining. We use static program analysis to find candidate call-sites suitable for monomorphization, and inline that call. Further, we analyze and optimize the program in multiple passes, to gain advantage from added precision after method inlining. Our analysis is object-sensitive inter-procedural points-to-analysis, which we use to find call-sites where receiver is a unique object, or a set with the same object type. These call-sites are candidates for monomorphization, which we inline.

Sometimes, we may not be able to uniquely resolve the call-site, but get a small number of target methods, which may be called at that site. For this, we split the call into multiple monomorphic calls, guarded with instance-of checks.

1.1 Real-World Impacts

Method Inlining is in itself a very useful optimization. Method calls involve pushing parameters on the stack, and jumping to the method code. These involve memory accesses, which slow down the execution. If method calls are inlined, there is no need to push parameters on the stack, speeding up the execution.

Moreover, method inlining enable other optimizations as well, like dead code elimination, constant propagation and other. So, when performing optimizations in multiple passes, a method inlining pass is very helpful. We perform only method inlining here, but we perform it in multiple passes so that we can perform multiple levels of inlining.

2 Static Analysis

We use an 1-object sensitive, flow and field sensitive, inter-procedural points to analysis. Heap object are abstracted by their allocation site and the context they are allocated in, so in a way we implement heap cloning. The algorithm is worklist based forward data flow analysis. Method calls are resolved using Soot's call graph information and our points to information. The context we analyse the methods in is the receiver object's identity (allocation site and its context).

Before

```
class A {
    A f;
    void foo() {
        f = new A();
        return 2;
    }
}

A a = new A();
int x = a.f();
```

After

```
A a = new A();
f_this = a;    // param copying
f_this.f = new A(); // function body
f_ret = 2;    // return
x = f_ret
```

Figure 1: Method Inlining

2.1 Handling Method Calls

Whenever we encounter a method call, we add the callee and its context to a worklist. We continuously poll from the worklist, and run analysis on the method in its context. For methods analyzed in a context, we merge all the in-PTGs from its callers, which introduces some imprecision that can be resolved with finer contexts. After the analysis of a method finishes, it triggers analysis of all its callers, until a fixed point is reached. Analysis of a method returns a points-to graph and a returned objects set, which we merge with the call-site's in-graph to obtain the call-site's out graph. So, points to information and information about returned object flows interprocedurally. Additionally, at each call-site, we maintain a list of possible targets, which is used by the optimizer to transform the code.

2.2 Assumptions Made

- We use only weak updates, which introduces a lot of imprecision. That is why we explore call-site splitting as an optimization.
- We do not consider dynamic language features like reflection
- We do not analyze library classes, as that leads to large analysis times and was intractable
- We do not inline recursive method calls

3 Transformations

We perform three kinds of transformations and inlining, each expanding the set of possible methods that can be inlined over the previous one.

3.1 Basic Method Inlining

This uses Class Hierarchy Analysis of Soot, and does not use any other information to resolve call sites. If Soot tells that a call-site has a unique target, this optimization phase inlines it.

Before

```
A a = ...

// a determined to be of class
// E by static analysis
a.foo()
```

After

```
A a = ...

E tmp = (E) a;
tmp.foo(); // inline E::foo here
```

Figure 2: Points-to Guided Inlining

Before

```
A a = ...

// a determined to be of
// classes B or C statically
a.foo()
```

After

```
A a = ...

if (a instanceof B) {
    tmp_B = (B) a;
    tmp_B.foo();
} else if (a instanceof C) {
    tmp_C = (C) a;
    tmp_C.foo();
} else {
    a.foo();
}
```

Figure 3: Call-site Splitting

3.2 Points-to Guided Inlining

In this phase, we use our analysis' points-to information, and call-site targets to monomorphize method calls which are not uniquely resolved by Soot's CHA. Say at a call site `a.foo()`, `a` may point to objects `{01, 02}`. If both `01.foo()` and `02.foo()` resolve to the same method, this phase inlines that method. We must introduce a cast statement here otherwise the `this` parameter copying would fail.

3.3 Call-site Splitting

Many times, we are not able to resolve a call site uniquely, but have multiple possible targets. These call sites are candidates for call-site splitting. This optimization has a tradeoff between optimization and code size, as if we have a very large number of possible targets, our code could easily blow up. So, we only split call sites which have upto 3 targets.

3.4 Challenges Found

- The necessity of casts. Even though the Jimple code we generated without the casts was valid, JVM would raise an error when we tried to access a field of a subclass from a variable of Base type. This was an error in points-to guided inlining, when we knew an object to be of

subclass' type, but its program type was the base class.

- We could not inline constructors. JVM raised an error saying constructor of wrong type. We could not resolve this, so we just skipped inlining constructors. However, since our analysis is intraprocedural and object sensitive, information is still not lost.
- We could not make any optimizations for recursive functions.

4 Results

We have created testcases which output the time it takes to run that testcase, and an integer result. To verify correctness of our transform, we compare the integer results. We have marked negative testcases with a *.

Testcase	Baseline (μs)	Optimized (μs)	Absolute Speedup (μs)	% Speedup
Test1	574853	123131	451722	78.58%
Test2	918199	456859	461340	50.24%
Test3	2252865	1589711	663154	29.43%
Test4	1945972	100411	1845561	94.84%
Test5	997107	862434	134673	13.51%
Test6*	1510537	1559734	-49197	-3%
Test7	558192	500145	58047	10.40%
Test8	339829	231806	108023	31.78%
Test9*	8144960	7907025	237935	2.92%
Test10	4602920	3121898	1481022	32.18%